Kịch Bản Video Cá Nhân

XLA Face Privacy System — Chi Tiết Từng Dòng Code

Môn Xử Lý Ảnh — Đồ Án Cuối Kỳ

Video	Người	Vai trò	Files trực tiếp viết
1	**Đạt**	Leader / Security	`privacy.py (269 dòng), clip_recorder.py (397 dò` `admin_auth.py (130 dòng), anonymizer_backend.py`
2	**Khoa**	Detection / Registry	`detector.py (149 dòng), face_recognition.py (69` `family_database.py (168 dòng), bbox_smoother.py`
3	**Khánh**	Backend / Pipeline	`app_api.py (1893 dòng), run_phone_cam.py (1314` `backend_api.py, frontend/ (6 trang HTML)`

Quy ước kịch bản: `dong X` = line number trong file gốc thực tế. Tất cả số dòng đã được verify bằng `grep -n`.

QUAN TRONG

Chuẩn bị trước khi quay: Server chạy cổng 8000 • VS Code font ≥16 • Browser 3 tab sẵn • Tắt notification • Test mic trước

Mục lục

1 VIDEO 1 — Đạt: Blur Engine + AES-256-GCM + PBKDF2 2

1.1 0:00–1:30 — Giới thiệu và khai báo đóng góp . . . 2
1.2 1:30–2:30 — Đặt vấn đề: Tại sao cần Dual Storage . . . 2
1.3 2:30–6:00 — Blur Engine: src/privacy.py . . . 2
1.4 6:00–9:30 — AES-256-GCM: src/clip_recorder.py . . . 6
1.5 9:30–12:00 — PBKDF2 + Timing Attack: src/admin_auth.py . . . 9
1.6 12:00 — Tóm tắt Video 1 . . . 11

2 VIDEO 2 — Khoa: YOLOv11 + InsightFace + FaceDB + BBoxSmoother 12

2.1 0:00–1:30 — Giới thiệu và khai báo đóng góp . . . 12
2.2 1:30–4:30 — YOLOv11n-face: src/detector.py . . . 12
2.3 4:30–7:30 — InsightFace: src/face_recognition.py . . . 14
2.4 7:30–10:00 — FamilyDatabase: src/family_database.py . . . 15
2.5 10:00–12:00 — BBox Smoother: src/bbox_smoother.py . . . 18

3 VIDEO 3 — Khánh: Backend + Pipeline + Frontend + Kết Luận 20

3.1 0:00–1:30 — Giới thiệu và khai báo đóng góp . . . 20
3.2 1:30–4:30 — FastAPI Architecture: src/app_api.py . . . 20
3.3 4:30–8:00 — Live Pipeline Architecture: app_api.py + run_phone_cam.py . . . 23
3.4 8:00–10:00 — 3 phương thức Streaming . . . 25
3.5 10:00–12:00 — Frontend 6 trang HTML . . . 27
3.6 12:00–13:30 — Demo end-to-end tổng hợp . . . 29
3.7 13:30–15:00 — Kết luận toàn bộ đề án . . . 29

4 Q&A — Câu Hỏi Dự Kiến + Đáp Án Chuẩn Bị 31

5 Checklist Kỹ Thuật Trước Khi Quay 32

1. VIDEO 1 — Đạt: Blur Engine + AES-256-GCM + PBKDF2

VIDEO 1 — DAT (LEADER) — Blur + Encryption + Auth

Camera rõ mặt, màn hình show code bên cạnh. Đạt phải nói rõ: "Em trực tiếp viết file này, cụ thể dòng X làm gì". Total code Đạt viết: ≈**1033 dòng** (269 + 397 + 130 + 237).

1.1 0:00–1:30 — Giới thiệu và khai báo đóng góp

[DAT — Leader] Giới thiệu

"Xin chào thầy cô. Em là Đạt, vai trò Leader của nhóm.
Phần em **trực tiếp viết code**:

- `src/privacy.py` — 269 dòng — toàn bộ Blur Engine 7 mode
- `src/anonymizer_backend.py` — 237 dòng — config system và 4 preset
- `src/clip_recorder.py` — 397 dòng — AES-256-GCM encrypt/decrypt clip
- `src/admin_auth.py` — 130 dòng — PBKDF2 + timing attack prevention
- `src/face_lock.py` — per-face encryption layer

Ngoài ra em thiết kế kiến trúc tổng thể và review code toàn nhóm."

1.2 1:30–2:30 — Đặt vấn đề: Tại sao cần Dual Storage

[DAT — Leader] Vấn đề bảo mật

"Camera quay tất cả mọi người — họ không đồng ý lưu khuôn mặt. Nguyên tắc **Privacy by Design** (GDPR Article 25) yêu cầu ẩn danh hoá mặc định.
Nhưng nếu chỉ lưu bản blur, bằng chứng điều tra bị mất. Giải pháp của em: lưu song song hai bản — **Bản blur** (`.mp4`) cho người dùng thông thường, **Bản gốc mã hoá** (`.enc`) chỉ admin với password mới mở được."

1.3 2:30–6:00 — Blur Engine: src/privacy.py

THAO TAC

Mở VS Code, mở `src/privacy.py`. Scroll qua nhanh cho thầy cô thấy file 269 dòng.

CODE — privacy.py dong 67–76 — _fast_gaussian_blur()

```
67 def _fast_gaussian_blur(roi: np.ndarray,
68                         blur_scale: float,
69                         blur_kernel: int) -> np.ndarray:
70     h, w = roi.shape[:2]
71     scale = min(max(blur_scale, 0.05), 1.0)
72     down_w = max(1, int(round(w * scale)))
73     down_h = max(1, int(round(h * scale)))
74     small = cv2.resize(roi, (down_w, down_h),
```

```
75                          interpolation=cv2.INTER_LINEAR)
76     small = cv2.GaussianBlur(small,
77                 (_odd(blur_kernel), _odd(blur_kernel)), 0)
78     return cv2.resize(small, (w, h),
79                       interpolation=cv2.INTER_LINEAR)  # dong 76
```

[DAT — Leader] Giải thích dong 67–76

"Dòng 67: hàm `_fast_gaussian_blur` nhận ROI (vùng khuôn mặt đã cắt ra), tham số scale và kernel size.
Dòng 71–72: downscale ROI xuống còn `scale%` kích thước. Preset balanced dùng scale mặc định ≈20%.
Dòng 73–74: GaussianBlur trên ảnh nhỏ — nhanh hơn blur trực tiếp trên ảnh lớn vì số pixel xử lý ít hơn nhiều.
Dòng 75–76: upscale lại kích thước gốc — kết quả là blur tự nhiên, mềm mại. Kỹ thuật này nhanh hơn GaussianBlur kernel lớn khoảng 3–5 lần."

THAO TAC

Scroll xuống dòng 78 file `src/privacy.py`:

CODE — privacy.py dong 78–86 — _pixelate()

```
78 def _pixelate(roi: np.ndarray, pixel_block: int) -> np.ndarray:
79     h, w = roi.shape[:2]
80     block = max(2, pixel_block)
81     small_w = max(1, w // block)
82     small_h = max(1, h // block)
83     # Compress xuong kich thuoc block -> mat chi tiet
84     small = cv2.resize(roi, (small_w, small_h),
85                        interpolation=cv2.INTER_LINEAR)
86     # Upscale INTER_NEAREST -> tao hieu ung mosaic vuong
87     return cv2.resize(small, (w, h),
88                       interpolation=cv2.INTER_NEAREST)  # dong 86
```

[DAT — Leader] Giải thích dong 78–86

"Dòng 78: `_pixelate` tạo hiệu ứng mosaic pixel. Dòng 84: downscale xuống còn 1/block kích thước. Dòng 86: **INTER_NEAREST** thay vì INTER_LINEAR — đây là chìa khoá. INTER_NEAREST không nội suy, mỗi pixel block vuông y hệt nhau, tạo ra hiệu ứng pixelate đặc trưng."

THAO TAC

Scroll xuống dòng 172 file `src/privacy.py` — quan trọng nhất:

CODE — privacy.py dong 172–200 — _obliterate_features() — mode manh nhat

```
172 def _obliterate_features(
173     roi: np.ndarray,
174     obliterate_scale: float,  # 0.06 theo preset "strong"
175     blur_kernel: int,
176     scramble_block: int,       # 10 theo preset "strong"
177     palette_levels: int,       # 8 theo preset "strong"
178     seed: int,
179 ) -> np.ndarray:
180     """Irreversible anonymization: destroy facial features."""
181     h, w = roi.shape[:2]
182     scale = min(max(obliterate_scale, 0.03), 0.25)
183     down_w = max(1, int(round(w * scale)))
184     down_h = max(1, int(round(h * scale)))
185
186     # Buoc 1 (dong 185): compress xuong 6% --- xoa mat, mui, mieng
187     work = cv2.resize(roi, (down_w, down_h),
188                       interpolation=cv2.INTER_AREA)
189     work = cv2.resize(work, (w, h),
190                       interpolation=cv2.INTER_NEAREST)
191
192     # Buoc 2 (dong 191): scramble block 10x10 --- pha vo hinh hoc
193     work = _scramble_blocks(work, block_size=scramble_block,
194                             seed=seed)
195
196     # Buoc 3 (dong 194): GaussianBlur >=35px --- xoa canh con lai
197     work = cv2.GaussianBlur(work,
198         (_odd(max(blur_kernel, 35)),
199          _odd(max(blur_kernel, 35))), 0)
200
201     # Buoc 4 (dong 197): quantize 8 mau --- xoa gradient tinh te
202     work = _quantize_colors(work, levels=palette_levels)
203     return work    # dong 200
```

[DAT — Leader] Giải thích chi tiet _obliterate_features

"Dòng 172–178: khai báo hàm với các tham số tùy chỉnh. Preset **strong** (anonymizer_backend.py dòng 70–79) truyền vào: `obliterate_scale=0.06`, `scramble_block=10`, `palette_levels=8`.

Bước 1 — dòng 185–188: Compress xuống còn 6% resolution. Ví dụ: khuôn mặt 100×100px → 6×6px → upscale lại thành 100×100px. Kết quả: toàn bộ mắt, mũi, miệng, đặc điểm nhận dạng biến mất hoàn toàn.

Bước 2 — dòng 191: `_scramble_blocks`: xáo trộn các block 10×10 pixel ngẫu nhiên với seed cố định. Phá vỡ quan hệ không gian giữa các vùng khuôn mặt. Dù bước 1 còn sót lại texture nhỏ, bước 2 xáo trộn hoàn toàn vị trí.

Bước 3 — dòng 194–197: GaussianBlur kernel ≥35×35 px. Xoá những cạnh sắc còn sót lại.

Bước 4 — dòng 197–198: Quantize về 8 màu. Loại bỏ gradient texture tinh tế mà các thuật toán nhận diện hiện đại có thể khai thác.
Kết quả: **không thể phục hồi bằng bất kỳ phương pháp nào**."

THAO TAC

Mở `src/anonymizer_backend.py`, scroll đến dòng 53. Show 4 preset cho thầy cô:

CODE — anonymizer_backend.py dong 53–91 — 4 Presets

```
53 PRESETS: Dict[PresetName, AnonymizationConfig] = {
54     "fast": AnonymizationConfig(      # dong 54
55         mode="pixelate",
56         imgsz=416,            # YOLO input nho hon -> nhanh hon
57         detect_every=3,       # detect 1 trong 3 frame -> 3x nhanh
               hon
58         proc_width=512,
59         pixel_block=18,
60     ),
61     "balanced": AnonymizationConfig(  # dong 62
62         mode="headcloak",
63         imgsz=512,
64         detect_every=2,        # detect 1 trong 2 frame
65         proc_width=640,
66         head_ratio=0.65,       # che 65% vung dau
67         face_padding=0.16,
68     ),
69     "strong": AnonymizationConfig(     # dong 70
70         mode="obliterate",     # mode manh nhat -> goi ham dong 172
71         imgsz=512,
72         detect_every=2,
73         obliterate_scale=0.06,  # compress xuong 6%
74         scramble_block=10,
75         palette_levels=8,
76     ),
77     "strict": AnonymizationConfig(     # dong 80
78         mode="headcloak",
79         imgsz=416,
80         detect_every=2,
81         head_ratio=0.95,    # che 95% vung dau (gan nhu toan bo)
82         face_padding=0.24,
83         blur_kernel=39,     # kernel lon hon -> blur manh hon
84     ),
85 }    # dong 91
```

[DAT — Leader] 4 Presets

"Dòng 53–91 trong `anonymizer_backend.py` là 4 preset em thiết kế. **fast** dòng 54: pixelate + YOLO 416 + detect 1/3 frame — cho máy yếu. **balanced** dòng 62: headcloak — che đầu, cân bằng tốc độ/bảo mật. **strong** dòng 70: obliterate — gọi hàm `_obliterate_features` dòng 172 vừa xem. **strict** dòng 80: headcloak cực mạnh, head_ratio 0.95 — che gần như toàn bộ đầu. Frontend gọi `GET /presets` (dòng 987–991 trong `app_api.py`) để lấy danh sách này về."

THAO TAC

Demo live: mở browser `http://localhost:8000/admin/dashboard.html`. Start pipeline, Đạt đứng trước camera. Lần lượt: fast → balanced → strong → strict. Đứng im để thầy cô thấy mức độ che giấu tăng dần.

1.4 6:00–9:30 — AES-256-GCM: src/clip_recorder.py

THAO TAC

Mở `src/clip_recorder.py`. Scroll nhanh: file 397 dòng. Dừng lại dòng 73.

CODE — clip_recorder.py dong 73–86 — _pack_frames(): serialise truoc khi ma hoa

```
73 def _pack_frames(frames: List[np.ndarray], fps: float) -> bytes:
74     """Serialise frames as length-prefixed JPEG stream."""
75     buf = io.BytesIO()
76     buf.write(struct.pack(">I", len(frames)))    # dong 76: 4B so
           luong frame
77     buf.write(struct.pack(">I", int(fps * 1000))) # dong 77: 4B fps
           *1000
78     for f in frames:
79         ok, enc = cv2.imencode(".jpg", f,
80             [cv2.IMWRITE_JPEG_QUALITY, 90])  # dong 80: JPEG
                   quality 90
81         if not ok:
82             continue
83         data = enc.tobytes()
84         buf.write(struct.pack(">I", len(data)))  # dong 84: 4B do
               dai frame
85         buf.write(data)                          # dong 85: JPEG
               bytes
86     return buf.getvalue()    # dong 86: tra ve toan bo stream bytes
```

[DAT — Leader] Giải thích _pack_frames dong 73–86

"Dòng 73: `_pack_frames` là bước chuẩn bị — serialize danh sách NumPy frame thành một khối bytes trước khi đưa vào AES encrypt.
Dòng 76–77: header gồm 4 byte số frame + 4 byte fps×1000 (integer arithmetic để tránh float). Dòng 80: mỗi frame encode thành JPEG quality 90 — đủ sắc nét, giảm size đáng kể. Dòng 84–85: mỗi frame ghi thêm 4 byte độ dài trước nội dung. Format này gọi là **length-prefixed stream** — tự mô tả, không cần delimiter."

THAO TAC

Scroll đến dòng 109:

CODE — clip_recorder.py dong 109–120 — encrypt_clip() — toan bo

```
109 def encrypt_clip(frames: List[np.ndarray],
110                  fps: float,
111                  password: str) -> bytes:
112     """
113     Encrypt frames with AES-256-GCM.
114     Returns: [ 32B salt ][ 12B nonce ][ ciphertext+tag ]
115     """
116     plaintext = _pack_frames(frames, fps)  # dong 114: serialize -> bytes
117     salt  = secrets.token_bytes(32)  # dong 115: per-clip random 256-bit salt
118     nonce = secrets.token_bytes(12)  # dong 116: per-clip random 96-bit nonce
119     key   = derive_clip_key(password, salt)  # dong 117: PBKDF2 -> AES-256 key
120     ct    = AESGCM(key).encrypt(nonce,       # dong 118: AES-GCM encrypt
121                                 plaintext, None)
122     return salt + nonce + ct  # dong 119: pack: 32B+12B+ciphertext
```

[DAT — Leader] Giải thích chi tiet từng dòng encrypt_clip

"**Dòng 114** — `_pack_frames`: gọi hàm vừa xem dòng 73, serialize frames thành plaintext bytes.
Dòng 115 — `secrets.token_bytes(32)`: sinh 32 byte ngẫu nhiên hoàn toàn (256-bit). Module `secrets` dùng OS entropy (CSPRNG), không phải `random.randbytes` thông thường.
Per-clip: mỗi clip có salt riêng — cùng password, key clip A $\neq$ key clip B.
Dòng 116 — `secrets.token_bytes(12)`: nonce 96-bit theo NIST SP 800-38D. 96-bit là chuẩn tốt nhất cho AES-GCM — cho phép 2^{96} nonce duy nhất.
Dòng 117 — `derive_clip_key`: gọi hàm dòng 124 trong `admin_auth.py`, tức là PBKDF2-SHA256 390.000 iterations — password ngắn → key 256-bit mạnh.
Dòng 118 — `AESGCM(key).encrypt`: đây là AES-256 trong chế độ GCM. GCM =

Galois/Counter Mode — authenticated encryption. Output = ciphertext + Authentication Tag 128-bit gộp vào cuối.
Dòng 119 — Pack file: `salt(32B) + nonce(12B) + ciphertext`. File `.enc` là **self-contained**: tất cả thông tin cần decrypt đều ở trong file."

THAO TAC

Scroll đến dòng 122:

CODE — clip_recorder.py dong 122–138 — decrypt_clip()

```
122 def decrypt_clip(data: bytes,
123                  password: str) -> Tuple[List[np.ndarray], float]:
124     """Decrypt an encrypted clip blob.
125     Raises ValueError on wrong password or corrupted data.
126     """
127     if len(data) < 44:              # dong 127: validate: 32+12=44 minimum
128         raise ValueError("Clip too short / corrupted.")
129     salt       = data[:32]          # dong 129: tach 32B salt dau
130     nonce      = data[32:44]        # dong 130: tach 12B nonce
131     ciphertext = data[44:]          # dong 131: phan con lai la ciphertext
132     key = derive_clip_key(password, salt)  # dong 132: re-derive key
133     try:
134         plaintext = AESGCM(key).decrypt(nonce, ciphertext, None)  # dong 134
135     except Exception:
136         # GCM auth tag fail: sai password HOAC file bi chinh sua
137         raise ValueError(
138             "Decryption failed -- wrong password or corrupted clip.")
139     return _unpack_frames(plaintext)   # dong 138: deserialize -> frames
```

[DAT — Leader] Giải thích decrypt_clip

"**Dòng 127**: guard clause check độ dài tối thiểu — 32+12=44 byte header.
Dòng 129–131: unpack file theo đúng format đã pack dòng 119. Salt, nonce đọc theo offset cố định — không cần delimiter.
Dòng 132: re-derive key — **cùng password + cùng salt = cùng key**. Đây là tính chất deterministic của PBKDF2.
Dòng 134: `AESGCM(key).decrypt` — đồng thời verify Authentication Tag. Nếu sai password → key sai → tag không khớp → exception. Nếu file bị sửa dù 1 bit → tag không khớp → exception. Exception được bắt và raise ValueError với message không tiết lộ chi tiết."

THAO TAC

Mở browser, tab `http://localhost:8000/admin/secure-archive.html`. Demo live:

1. Chỉ danh sách file `.enc` trong `data/clips/secure/`
2. Click một clip, click **Decrypt & Play**
3. Nhập đúng password — clip gốc rõ nét hiện ra
4. Nhập sai password — error "Decryption failed" (message từ dòng 136)

1.5 9:30–12:00 — PBKDF2 + Timing Attack: src/admin_auth.py

THAO TAC

Mở `src/admin_auth.py`. Dừng ở dòng 31.

CODE — admin_auth.py dong 31–50 — _ITERATIONS, _pbkdf2, _ct_equal

```
31 _ITERATIONS = 390_000     # OWASP 2023 minimum for PBKDF2-SHA256
32 _KEY_LEN      = 32        # 256-bit output
33 _DEFAULT_CRED_PATH = "data/admin_credentials.json"
34
35 def _pbkdf2(password: str, salt: bytes,
36             iterations: int = _ITERATIONS) -> bytes:
37     kdf = PBKDF2HMAC(
38         algorithm=hashes.SHA256(),
39         length=_KEY_LEN,           # 32 bytes
40         salt=salt,
41         iterations=iterations,     # dong 41: 390.000 lan SHA-256
42     )
43     return kdf.derive(password.encode("utf-8"))  # dong 43
44
45 def _ct_equal(a: bytes, b: bytes) -> bool:
46     """Constant-time comparison to prevent timing attacks."""
47     return hmac.compare_digest(a, b)  # dong 50
```

[DAT — Leader] Giải thích dong 31–50

"**Dòng 31**: `_ITERATIONS = 390_000` — con số này không phải em tự nghĩ ra. Đây là chuẩn **OWASP 2023** cho PBKDF2-SHA256. OWASP (Open Web Application Security Project) là tổ chức bảo mật uy tín nhất.

Dòng 41: 390.000 iterations nghĩa là SHA-256 được tính 390.000 lần liên tiếp cho *mỗi lần* verify password. Máy tính 1 triệu hash/giây → mỗi verify mất 0.39 giây. Brute-force 8 ký tự mixed = $96^8 \approx 7 \times 10^{15}$ khả năng × 0.39s = hàng triệu năm.

Dòng 48–50: `_ct_equal` dùng `hmac.compare_digest`.

Tại sao không dùng `a == b`? Python `==` so sánh byte từng ký tự, dừng lại ngay khi gặp byte đầu tiên khác. Hacker đo thời gian server phản hồi: response nhanh → sai ngay byte đầu, response chậm hơn → 5 byte đầu đúng. Qua 1000 request, thống kê timing → leak thông tin về password — **Timing Attack**.

`hmac.compare_digest` **luôn** chạy hết tất cả bytes bất kể sai ở đâu. Response time không đổi → không có thông tin rò rỉ."

THAO TAC

Scroll đến dòng 83:

CODE — admin_auth.py dong 83–94 — verify_admin(): flow day du

```
83 def verify_admin(password: str,
84                  path: str = _DEFAULT_CRED_PATH) -> bool:
85     """Returns True iff password matches stored credentials."""
86     p = Path(path)
87     if not p.exists():
88         return False
89     try:
90         data = json.loads(p.read_text(encoding="utf-8"))
91         salt      = base64.b64decode(data["salt"])     # dong 91
92         stored    = base64.b64decode(data["hash"])     # dong 92
93         iters     = int(data.get("iterations",         # dong 93
94                                  _ITERATIONS))
95         candidate = _pbkdf2(password, salt, iters)     # dong 94
96         return _ct_equal(candidate, stored)            # dong 95
97     except Exception:
98         return False    # dong 97: fail silently, no detail leak
```

[DAT — Leader] Giải thích verify_admin

"**Dòng 91**: đọc salt từ file JSON. Salt được lưu dạng base64 — không phải plaintext, không phải hex.
Dòng 94: `_pbkdf2(password, salt, iters)` — tức cùng password + cùng salt → cùng hash. Đây là tính deterministic của PBKDF2.
Dòng 95: so sánh constant-time (hàm dòng 48 vừa xem).
Dòng 97: `except Exception: return False` — lỗi JSON parse, lỗi base64, lỗi bất kỳ đều trả False. Không bao giờ trả exception có detail về lỗi ở đâu. Đây là nguyên tắc *fail silently, no information disclosure*."

THAO TAC

Mở `data/admin_credentials.json` trong editor. Show cho thầy cô thấy:

[DAT — Leader] Zero knowledge of plaintext

"File này chỉ có `salt` và `hash` dạng base64. Không có password. Không có key. **Ngay cả em viết code cũng không biết password admin** nếu không được admin cung cấp trực tiếp. Đây là nguyên tắc *Zero Knowledge of Plaintext* — privacy by design thực sự."

1.6 12:00 — Tóm tắt Video 1

[DAT — Leader] Ket thuc video 1

"Tóm lại, code em trực tiếp viết: `privacy.py` dòng 67–200 — 7 mode ẩn danh, `anonymizer_backend.py` dòng 53–91 — 4 preset, `clip_recorder.py` dòng 109–138 — AES-256-GCM encrypt/decrypt, `admin_auth.py` dòng 31–97 — PBKDF2 + timing attack prevention.
Tất cả hướng đến một mục tiêu: bảo vệ quyền riêng tư với tiêu chuẩn bảo mật công nghiệp. Cảm ơn thầy cô."

2. VIDEO 2 — Khoa: YOLOv11 + InsightFace + FaceDB + BBoxSmoother

VIDEO 2 — KHOA (Detection) — YOLO + InsightFace + FaceDB

Camera rõ mặt, VS Code mở sẵn đúng file. Khoa phải nói rõ: "Dòng X em viết để làm gì, tại sao chọn giá trị đó".
Total code Khoa viết: ≈**759 dòng** (149 + 69 + 168 + 373).

2.1 0:00–1:30 — Giới thiệu và khai báo đóng góp

[KHOA — Detection] Tu gioi thieu

"Xin chào thầy cô. Em là Khoa, phụ trách **Face Detection và Recognition**.
Phần em **trực tiếp viết**:

- `src/detector.py` — 149 dòng — FaceDetector wrapper cho YOLOv11n-face
- `src/face_recognition.py` — 69 dòng — InsightFace buffalo_l embedding 512-dim
- `src/family_database.py` — 168 dòng — FamilyDatabase, matching engine
- `src/bbox_smoother.py` — 373 dòng — BBoxSmoother EMA + IoU tracking

Ngoài ra em thu thập dataset gốc — chụp 200 ảnh trong nhiều điều kiện ánh sáng."

2.2 1:30–4:30 — YOLOv11n-face: src/detector.py

THAO TAC

Mở VS Code, mở `src/detector.py`. Nhìn qua: 149 dòng. Dừng dòng 13.

CODE — detector.py dong 13–36 — class FaceDetector __init__()

```
13 class FaceDetector:
14     """Face detector backed by Ultralytics YOLOv11-face."""
15
16     def __init__(
17         self,
18         model_path: str    = "models/yolov11n-face.pt",
19         conf_threshold: float = 0.35,   # dong 19
20         iou_threshold:  float = 0.50,   # dong 20
21         imgsz:          int   = 640,    # dong 21
22         device: Optional[str] = None,
23     ) -> None:
24         self.conf_threshold = conf_threshold   # dong 34
25         self.iou_threshold  = iou_threshold    # dong 35
26         self.imgsz          = imgsz             # dong 36
27         self.model_path     = model_path
28         self.model: Optional[Any] = None
29
30         try:
31             yolo_cls = self._resolve_yolo_class()  # dong 43:
                   import YOLO
```

```
32            if yolo_cls is None:
33                return
34            self.model = yolo_cls(model_path)      # dong 45: load
                weights
35        except Exception as exc:
36            LOGGER.exception("Failed to load: %s", exc)
```

[KHOA — Detection] Giai thich dong 13–36

"**Dòng 18**: model path mặc định `models/yolov11n-face.pt` — file 6.2MB weights. Đây là model fine-tuned cho face detection, không phải YOLO thông thường. "n" = nano: nhỏ nhất trong họ YOLOv11, chạy tốt trên CPU.
Dòng 19: `conf_threshold = 0.35`. Em chạy benchmark với nhiều giá trị: `0.25` → quá nhiều false positive (nhận ra màn hình TV, ảnh treo tường là mặt người). `0.5` → bỏ sót mặt khi ánh sáng yếu. `0.35` cân bằng tốt nhất giữa precision và recall.
Dòng 20: `iou_threshold = 0.50`. IoU (Intersection over Union) dùng cho Non-Maximum Suppression. Khi hai bbox chồng lên nhau >50%, giữ cái có confidence cao nhất, bỏ cái kia. Tránh detect cùng khuôn mặt hai lần.
Dòng 21: `imgsz = 640`. YOLO resize mọi frame về 640×640 trước inference. Chuẩn hoá input, model không bị confused với kích thước khác nhau.
Dòng 45: `self.model = yolo_cls(model_path)` — load weights vào RAM. Lần đầu ≈1–2 giây, sau đó cached."

THAO TAC

Scroll đến dòng 89:

CODE — detector.py dong 89–135 — detect_faces_with_scores() — core function

```
89  def detect_faces_with_scores(
90      self, frame: np.ndarray
91  ) -> List[Tuple[List[int], float]]:
92      ...
93      results = self.model.predict(   # dong 113: call YOLO inference
94          source=frame,
95          conf=self.conf_threshold,   # dong 115: nguong tu dong 19
96          iou=self.iou_threshold,     # dong 116: NMS tu dong 20
97          imgsz=self.imgsz,           # dong 117: 640 tu dong 21
98          device=self.device,
99          verbose=False,              # dong 119: tat YOLO log
100     )
101     ...
102     for idx, xyxy in enumerate(boxes.xyxy): # dong 131: duyet tung
            box
103         x1, y1, x2, y2 = xyxy.tolist()      # dong 132: toa do goc
                tren-trai, duoi-phai
104         # Clamp: dam bao trong frame
```

```
        x1_i = max(0, min(width - 1, int(round(x1))))
        y1_i = max(0, min(height - 1, int(round(y1))))
        x2_i = max(0, min(width,      int(round(x2))))
        y2_i = max(0, min(height,     int(round(y2))))
        w_box = x2_i - x1_i
        h_box = y2_i - y1_i
        # Convert [x1,y1,x2,y2] -> [x,y,w,h] format
        detections.append(([x1_i, y1_i, w_box, h_box], conf_val))
```

[KHOA — Detection] Giai thich dong 131 convert format

"**Dòng 131–132**: YOLO output format là `[x1,y1,x2,y2]` — tọa độ góc trên trái và góc dưới phải.
Dòng 133–142: em clamp về trong boundary của frame. Không clamp có thể gây negative width/height khi detect gần mép frame.
Cuối function: convert về `[x,y,w,h]`. Lý do: OpenCV `cv2.rectangle` + privacy.py `_expand_box` đều dùng format `[x,y,w,h]`. Convert một lần ở đây, toàn bộ pipeline dùng format nhất quán."

2.3 4:30–7:30 — InsightFace: src/face_recognition.py

THAO TAC

Mở `src/face_recognition.py`. File 69 dòng — nhỏ nhưng quan trọng.

CODE — face_recognition.py dong 10–24 — FaceRecognizer + cosine_similarity

```
class FaceRecognizer:
    """InsightFace embedding extractor for detected face boxes."""

    def __init__(self, model: str = "buffalo_l",
                 det_size: tuple[int, int] = (256, 256)) -> None:
        self.model = model                    # dong 14: ghi lai ten model
        self.app = FaceAnalysis(              # dong 15
            name=model,
            providers=["CPUExecutionProvider"]  # dong 17: chi dung CPU
        )
        self.app.prepare(ctx_id=0,
                         det_size=det_size) # dong 18: 256x256 ROI

    @staticmethod
    def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
        a_norm = np.linalg.norm(a)            # dong 20: ||a||
        b_norm = np.linalg.norm(b)            # dong 21: ||b||
        if a_norm == 0.0 or b_norm == 0.0:
```

```
28            return -1.0                          # dong 23: edge case
                zero vector
29        return float(np.dot(a, b) /              # dong 24
30                     (a_norm * b_norm))          # cos(a,b) = (a.b)/(||a
                          ||.||b||)
```

[KHOA — Detection] Giai thich face_recognition.py

"**Dòng 15–17**: `FaceAnalysis(name="buffalo_l")` — đây là model InsightFace. "buffalo_l" là model lớn nhất trong bộ InsightFace free, đạt 99.7% trên benchmark LFW (Labeled Faces in the Wild). Output: embedding vector 512 chiều.
Dòng 17: `CPUExecutionProvider` — chạy trên CPU, không cần GPU. Tốc độ ≈50–80ms/frame, đủ cho camera giám sát.
Dòng 20–24: Cosine Similarity công thức:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

Result từ -1 (ngược hướng hoàn toàn) đến $+1$ (cùng người). **Tại sao cosine không phải Euclidean distance?** Cosine không phụ thuộc độ lớn vector — chỉ so sánh hướng. Cùng người trong ảnh sáng vs ảnh tối: magnitude khác nhau nhưng hướng gần nhau. Euclidean distance bị ảnh hưởng bởi magnitude → kém ổn định hơn."

2.4 7:30–10:00 — FamilyDatabase: src/family_database.py

THAO TAC

Mở `src/family_database.py`. Dừng dòng 17.

CODE — family_database.py dong 17–34 — FamilyMember dataclass + FamilyDatabase

```
17 @dataclass
18 class FamilyMember:
19     member_id:  str                          # dong 18: ID duy nhat (
           khong doi)
20     name:       str                          # dong 19: ten hien thi
21     embeddings: List[List[float]]            # dong 20: N x 512 float
           matrix
22     threshold:  float = 0.35                 # dong 21: per-member
           threshold
23
24 @dataclass
25 class FamilyDatabase:
26     def __init__(self,
27                  db_path: str = "data/family_members.json") -> None
                     :
```

```
28          self._emb_matrix = np.empty((0,0), dtype=np.float32)   # dong 30
29          self._member_ids: List[str] = []
30          self._member_thresholds = np.empty((0,), dtype=np.float32)
31          self.load()    # dong 34: tu dong load khi khoi tao
```

[KHOA — Detection] FamilyMember schema

"**Dòng 17–21**: `FamilyMember` là dataclass — struct đơn giản không có method.
Dòng 20: `embeddings: List[List[float]]` là danh sách các vector 512 chiều. Mỗi thành viên có thể có N embedding từ N ảnh mẫu. Em thu thập khoảng 100 ảnh/người ở nhiều góc độ, ánh sáng.
Dòng 21: `threshold = 0.35` là per-member. Người có khuôn mặt "khó nhận" (đeo kính dày, ánh sáng bất thường) admin có thể hạ threshold xuống 0.30. **Dòng 30**: `_emb_matrix` là numpy matrix pre-built — tất cả embedding của tất cả thành viên được stack thành 1 ma trận lớn. Để match vectorized, không dùng Python loop."

THAO TAC

Scroll đến dòng 101:

CODE — family_database.py dong 101–128 — _rebuild_index(): vectorized matching prep

```
101 def _rebuild_index(self) -> None:
102     flat_embeddings: List[np.ndarray] = []
103     ...
104     for member in self.members:
105         for emb_list in member.embeddings:    # dong 109: duyet tung mau
106             emb = np.asarray(emb_list, dtype=np.float32)
107             norm = float(np.linalg.norm(emb))
108             if norm <= 0.0:
109                 continue
110             flat_embeddings.append(emb / norm)  # dong 116: L2-normalize truoc
111             member_ids.append(member.member_id)
112
113     # Stack toan bo thanh ma tran [N_total_samples x 512]
114     self._emb_matrix = np.stack(flat_embeddings).astype(np.float32)
        # dong 128
```

THAO TAC

Scroll xuống dòng 139:

CODE — family_database.py dong 139–166 — match(): vectorized cosine similarity

```
139 def match(self, query_embedding: np.ndarray,
140           threshold: float = 0.35
141           ) -> Optional[Tuple[str, str, float]]:
142     if self._emb_matrix.size == 0:
143         return None     # dong 144: database rong -> khong nhan dien
144
145     q = np.asarray(query_embedding, dtype=np.float32)
146     q_norm = float(np.linalg.norm(q))
147     q = q / q_norm                      # dong 155: normalize query
148
149     scores = self._emb_matrix @ q       # dong 156: VECTORIZED dot
        product
150     # Viet ra: scores[i] = emb_matrix[i] . q
151     #          = cos(emb[i], q) vi ca hai da duoc L2-normalize truoc
        (dong 116)
152
153     thresholds = np.maximum(            # dong 157: per-member
        threshold
154                     self._member_thresholds, float(threshold))
155     valid = scores >= thresholds        # dong 158: boolean mask
156
157     masked_scores = np.where(valid, scores, -np.inf)  # dong 162
158     best_idx    = int(np.argmax(masked_scores))        # dong 163:
        tim max
159     best_score  = float(masked_scores[best_idx])       # dong 164
160
161     return (self._member_ids[best_idx],
162             self._member_names[best_idx],
163             best_score)                 # dong 166: (id, ten, score)
```

[KHOA — Detection] Giai thich match() vectorized

"**Dòng 116** (rebuild_index): Trước khi lưu vào matrix, mọi embedding được **L2-normalize**. Sau normalize: `||emb|| = 1`.

Dòng 156: `scores = self._emb_matrix @ q`. Đây là matrix-vector multiplication. `_emb_matrix` shape `[N, 512]`, `q` shape `[512]`. Kết quả `scores` shape `[N]` — similarity với tất cả N mẫu **trong một lần tính**.

Tại sao đây là cosine similarity? Vì cả query (dòng 155) và database (dòng 116) đều đã L2-normalize. Với $\|a\| = 1$ và $\|b\| = 1$: $\cos(a, b) = a \cdot b$ — dot product là cosine. Không cần chia thêm $\|a\| \cdot \|b\|$ nữa.

Tại sao vectorized quan trọng? Database 2 người × 100 ảnh = 200 mẫu. Python loop: 200 iterations, mỗi cái gọi `np.dot` riêng lẻ. Matrix multiply: 200 phép tính **song song**, BLAS/LAPACK tối ưu, nhanh hơn 10–20x."

2.5 10:00–12:00 — BBox Smoother: src/bbox_smoother.py

THAO TAC

Mở `src/bbox_smoother.py`. File 373 dòng, phần core từ dòng 24.

CODE — bbox_smoother.py dong 24–50 — BBoxSmoother __init__

```
24 class BBoxSmoother:
25     """
26     Temporal smoothing for bboxes using EMA.
27     Without smoothing: boxes jump per-frame (YOLO detects slightly
           different).
28     EMA alpha=0.7 -> responsive but smooth.
29     """
30
31     def __init__(self,
32                  alpha: float = 0.7,          # dong 33: smoothing
                        factor
33                  iou_threshold: float = 0.3   # dong 35: matching
                        threshold
34                  ) -> None:
35         self.alpha          = max(0.1, min(1.0, alpha))    # dong 47
36         self.iou_threshold = max(0.1, min(0.9,
37                                           iou_threshold)) # dong 49
```

THAO TAC

Scroll xuống dòng 115:

CODE — bbox_smoother.py dong 115–132 — _ema_smooth(): EMA formula

```
115 def _ema_smooth(self, current: List[int],
116                 history: deque) -> List[int]:
117     """Apply exponential moving average smoothing."""
118     current_arr = np.array(current, dtype=float)
119     if not history:
120         return current          # don't smooth, no history yet
121
122     smoothed = current_arr      # dong 124: start with current
123     weight = self.alpha         # dong 125: weight = 0.7 for current
124
125     for past in reversed(history):  # dong 127: oldest first
126         past_arr = np.array(past, dtype=float)
127         # EMA formula:
128         # smoothed = weight * smoothed + (1-weight) * past
129         smoothed = weight * smoothed + (1 - weight) * past_arr   #
                dong 129
130         weight *= self.alpha  # dong 130: decay: 0.7^2, 0.7^3...
```

```
131
132     return [int(round(v)) for v in smoothed]  # dong 132
```

[KHOA — Detection] Giai thich EMA

"**EMA — Exponential Moving Average**:

$$\hat{b}_t = \alpha \cdot b_t + (1 - \alpha) \cdot \hat{b}_{t-1}$$

Dòng 125: alpha = 0.7 cho frame hiện tại. **Dòng 130**: weight của frame $t-1$ là $0.7^2 = 0.49$, frame $t-2$ là $0.7^3 = 0.343$, v.v. Frame càng cũ càng ít ảnh hưởng — *exponential* decay.
Tại sao không dùng simple moving average? SMA cho trọng số bằng nhau cho tất cả frame trong window. Box thay đổi đột ngột (người chạy nhanh) → SMA lag theo không kịp. EMA: frame mới nhất quan trọng nhất → responsive + smooth.
Kết quả thực tế (dòng 35): `iou_threshold = 0.3` cho matching boxes qua frames. Nếu box mới overlap >30% với track cũ → match, apply EMA. Dưới 30% → người mới đi vào frame, tạo track mới."

THAO TAC

Demo live: mở browser, start pipeline mode balanced. Di tay từ từ trước khuôn mặt. Box bám theo mượt mà. Rồi thay bằng mode fast không smoother (nếu có toggle) — box nhảy.

[KHOA — Detection] Ket thuc video 2

"Tóm lại code em viết: `detector.py` dòng 13–36 init, dòng 89–145 detect, `face_recognition.py` dòng 10–56, `family_database.py` dòng 17–166 include vectorized match, `bbox_smoother.py` dòng 24–132 EMA smoother.
Ngoài ra em thu thập dữ liệu — 200 ảnh đăng ký vào database. Cảm ơn thầy cô."

3. VIDEO 3 — Khánh: Backend + Pipeline + Frontend + Kết Luận

VIDEO 3 — KHANH (CUOI) — FastAPI + Pipeline + Frontend + KET LUAN

Video cuối cùng. Khánh trình bày backend, pipeline, streaming, frontend và **kết luận toàn bộ đề án**. Total code Khánh viết: **≈3207 dòng** (1893 + 1314).

3.1 0:00–1:30 — Giới thiệu và khai báo đóng góp

[KHANH — Backend] Tu gioi thieu

"Xin chào thầy cô. Em là Khánh, phụ trách **Backend API và Real-time Pipeline**. Và em là người trình bày **cuối cùng** của nhóm.
Phần em **trực tiếp viết**:

- `src/app_api.py` — **1893 dòng** — toàn bộ REST API + WebSocket
- `src/run_phone_cam.py` — **1314 dòng** — FrameReader thread, camera utils
- `src/backend_api.py` — stateless endpoint handlers (image processing)
- `frontend/` — **6 trang HTML** với Tailwind CSS

Nói ngắn gọn: Đạt làm tầng bảo mật, Khoa làm tầng nhận diện, em làm *"dây nối"* — bọc tất cả vào API, serve qua HTTP, giao diện người dùng."

3.2 1:30–4:30 — FastAPI Architecture: src/app_api.py

THAO TAC

Mở VS Code, mở `src/app_api.py`. Scroll nhanh từ đầu: 1893 dòng. Zoom out nhìn outline — show thầy cô thấy scale. Sau đó jump đến dòng 968.

CODE — app_api.py dong 968–993 — Nhom 1: Info/Health endpoints

```
968 @app.get("/health", tags=["Info"], summary="Service health check")
969 def health() -> Dict[str, Any]:            # dong 969
970     ...
971
972 @app.get("/info", tags=["Info"], summary="Service capabilities")
973 def info() -> Dict[str, Any]:              # dong 974
974     ...
975
976 @app.get("/presets", tags=["Info"],
977          summary="List all anonymization presets")
978 def get_presets() -> Dict[str, Any]:       # dong 988
979     ...
980 # -> goi tiep sang anonymizer_backend.py dong 53 (PRESETS dict cua
    Dat)
981
982 @app.get("/pipeline/modes", tags=["Info"])
983 def pipeline_modes() -> Dict[str, Any]:    # dong 993
```

```
984     ...
```

[KHANH — Backend] Nhom Info endpoints

"**Dòng 968–993**: 4 endpoint nhóm Info. `GET /health` dòng 969: server alive check, monitor dùng. `GET /info` dòng 974: version, capabilities — frontend fetch khi load. `GET /presets` dòng 988: gọi tiếp `PRESETS` dict dòng 53 của Đạt trong `anonymizer_backend.py`. Đây là ví dụ cụ thể về tích hợp giữa module của em và module của Đạt."

THAO TAC

Scroll đến dòng 1005:

CODE — app_api.py dong 1005–1102 — Nhom 2: Stateless image processing

```
1005 @app.post(...)                          # dong 1005
1006 def anonymize(request: FrameRequest) -> Dict[str, Any]:   # dong
     1010
1007     # Nhan base64 image -> blur mat -> tra ve base64 image
1008
1009 @app.post(...)                          # dong 1030
1010 def lock(request: LockRequest) -> Dict[str, Any]:         # dong
     1035
1011     # Nhan image + password -> AES-GCM ma hoa tung vung mat
1012
1013 @app.post(...)                          # dong 1069
1014 def unlock(request: UnlockRequest) -> Dict[str, Any]:     # dong
     1074
1015     # Nhan image + per-face encrypted data -> decrypt -> hien thi
1016
1017 @app.post(...)                          # dong 1096
1018 def recognize(request: RecognizeRequest) -> RecognizeResponse: #
     dong 1102
1019     # Nhan image -> tra ve ten tat ca nguoi nhan ra duoc
```

[KHANH — Backend] Nhom Stateless endpoints

"Dòng 1005–1102: 4 endpoint xử lý ảnh tĩnh, không cần pipeline. Tất cả **stateless** — mỗi request độc lập, không phụ thuộc state server.
`POST /anonymize` dòng 1010: nhận base64 ảnh, gọi `anonymize_faces` từ `privacy.py` của Đạt. `POST /lock` dòng 1035: gọi `encrypt_clip` logic từ `face_lock.py` của Đạt. `POST /recognize` dòng 1102: gọi `FaceRecognizer` của Khoa + database matching.
Em bọc logic của Đạt và Khoa vào REST interface — đây là vai trò của backend integration."

THAO TAC

Scroll đến dòng 1427:

CODE — app_api.py dong 1427–1500 — Nhom 3: Pipeline control endpoints

```
1427 @app.post(...)
1428 def pipeline_start(req: PipelineStartRequest) -> Dict[str, Any]: # dong 1432
1429     # Tao PipelineSession moi, goi .start()
1430     # PipelineSession la class em viet dong 512
1431
1432 @app.delete(...)
1433 def pipeline_stop() -> Dict[str, Any]:        # dong 1458
1434     # Goi session.stop() -> join thread
1435
1436 @app.get(...)
1437 def pipeline_status() -> PipelineStatusResponse:  # dong 1474
1438     # Lay real-time stats: fps, faces_detected, mode hien tai
1439
1440 @app.patch("/pipeline/config", tags=["Pipeline"],
1441            summary="Hot-update settings without restarting")
1442 def pipeline_update_config(patch: PipelineConfigPatch): # dong 1488
1443     with _pipeline_lock:                # dong 1496: thread-safe
1444         session = _pipeline_session
1445     if session is None or not session.is_alive:
1446         raise HTTPException(status_code=404, ...)
1447     session.patch_config(patch)         # dong 1500: goi PipelineSession.patch_config
1448     applied = patch.model_dump(exclude_none=True)
1449     return {"ok": True, "applied": applied}
```

[KHANH — Backend] Hot-reload config

"**Dòng 1483–1501**: đây là tính năng **hot-reload config** em thiết kế.
`PATCH /pipeline/config` cho phép đổi mode, threshold, detect_every **mà không cần restart pipeline**.
Dòng 1496: `with _pipeline_lock` — lock trước khi access shared session. Thread-safe: processing thread đang dùng config cũ sẽ không bị race condition.
Dòng 1500: `session.patch_config(patch)` nhảy vào dòng 583 của class `PipelineSession`. Xem dòng 583–588 tiếp:"

CODE — app_api.py dong 583–588 — PipelineSession.patch_config(): atomic update

```
583 def patch_config(self, patch: PipelineConfigPatch) -> None:
584     with self.cfg_lock:             # dong 584: lock cfg_lock rieng
585         for key, value in patch.model_dump(
```

```
586                 exclude_none=True).items():  # dong 585: chi fields
                        khong None
587             if hasattr(self.live_cfg, key):
588                 setattr(self.live_cfg, key, value)  # dong 587: set
                        attribute
589     # cfg_lock release -> processing thread thay doi ngay lap tuc
```

[KHANH — Backend] Atomic config update

"**Dòng 583–588**: `PipelineSession.patch_config` nằm trong class em viết dòng 512.
Dòng 584: `with self.cfg_lock` — lock riêng cho config, khác với lock cho pipeline session tổng thể. Granular locking: processing thread lock config nhẹ hơn session lock.
Dòng 585: `model_dump(exclude_none=True)` — chỉ lấy fields mà client gửi lên. Nếu PATCH request chỉ đổi mode, chỉ mode thay đổi, các field khác giữ nguyên.
Dòng 587: `setattr(self.live_cfg, key, value)` — set ngay trên live config object. Processing loop dòng 636 đọc `self.live_cfg` mỗi frame iteration → thay đổi có hiệu lực ngay frame tiếp theo."

3.3 4:30–8:00 — Live Pipeline Architecture: app_api.py + run_phone_cam.py

THAO TAC

Mở `src/run_phone_cam.py`. Jump đến dòng 327.

CODE — run_phone_cam.py dong 327–386 — FrameReader thread

```
327 class FrameReader(threading.Thread):
328     """Read frames on a dedicated thread;
329        keep only the LATEST frame in a maxsize=1 queue."""
330
331     def __init__(self, cap: cv2.VideoCapture) -> None:
332         super().__init__(daemon=True)    # dong 331: daemon thread
333         self.cap    = cap
334         self.queue: Queue = Queue(maxsize=1)  # dong 333: only 1
                slot!
335         self.running = True
336         self.frames_read    = 0
337         self.frames_failed = 0
338
339     def run(self) -> None:                    # dong 338: thread
            entry
340         while self.running:
341             ok, frame = self.cap.read()       # dong 341: BLOCKING
                    call
342             ...
343             # Queue full? Pop old frame truoc khi push moi
344             if self.queue.full():             # dong 365
```

```
345                 try:
346                     self.queue.get_nowait()    # dong 367: discard
                           stale
347                 except Empty:
348                     pass
349             self.queue.put_nowait((ok, frame)) # dong 369: push
                   latest
350
351     def read(self, timeout: float = 0.6):      # dong 376
352         try:
353             return self.queue.get(timeout=timeout)  # dong 378
354         except Empty:
355             return False, None                       # dong 380
356
357     def stop(self) -> None:                    # dong 382
358         self.running = False
```

[KHANH — Backend] Thiet ke FrameReader

"**Dòng 327**: `FrameReader` là class em viết trong `run_phone_cam.py`. Class này được dùng trực tiếp bởi `PipelineSession._run()` dòng 711 trong `app_api.py`.
Dòng 341: `self.cap.read()` là blocking call. `cv2.VideoCapture.read()` đợi cho đến khi camera deliver frame mới. Nếu call này trong thread xử lý chính: trong khi YOLO inference (50–100ms), thread block, camera buffer tràn, frame cũ bị drop.
Dòng 333: `Queue(maxsize=1)` — queue chỉ có **1 slot**. Thiết kế cố ý: chỉ giữ lại frame mới nhất.
Dòng 365–369: nếu queue đã full (processing thread chưa kịp lấy), `get_nowait()` pop frame cũ ra, rồi `put_nowait` push frame mới vào. Kết quả: processing thread luôn nhận frame **thực sự mới nhất**.
Dòng 376–380: `read()` với timeout 0.6s. Nếu 0.6 giây không có frame mới → trả `(False, None)`. Pipeline loop sẽ skip frame này, không crash."

THAO TAC

Mở `src/app_api.py`, jump đến dòng 636:

CODE — app_api.py dong 636–742 — PipelineSession._run(): main loop

```
636 def _run(self) -> None:
637     ...
638     # Buoc 1: resolve camera source (dong 638--669)
639     cap = open_capture(source, backend_flag, ...)    # dong 672: mo
           camera
640
641     # Buoc 2: khoi tao modules (dong 673--705)
642     detector = FaceDetector(              # dong 674: cua Khoa
643         model_path=req.model_path,
```

```
644             conf_threshold=req.conf_threshold, ...)
645         _db = FamilyDatabase(db_path=req.face_db)  # dong 688: cua Khoa
646         _rec = FaceRecognizer(model=req.recognition_model)  # dong 692:
                cua Khoa
647
648         # Buoc 3: khoi dong FrameReader thread (dong 710--729)
649         frame_reader = FrameReader(cap)      # dong 711: class dong 327
650         frame_reader.start()                 # dong 712: bat dau doc
               camera
651
652         # Buoc 4: vong lap chinh (dong 730+)
653         while not self._stop_event.is_set():
654             ok, frame = frame_reader.read() # dong 744: lay frame moi
                   nhat
655             if not ok or frame is None:
656                 continue
657
658             # Chi detect sau moi detect_every frame (tiet kiem CPU)
659             if frame_idx % cfg.detect_every == 0:  # dong 750
660                 boxes, scores = detector.detect_faces_with_scores(
                       inference)
661                 ...
662             # Recognition, smoothing, anonymize, stream...
```

[KHANH — Backend] Giai thich pipeline loop

"**Dòng 636**: `_run()` là method của class `PipelineSession` em viết dòng 512. Đây là hàm chạy trong background thread khi admin click Start Pipeline.
Dòng 674–692: em khởi tạo `FaceDetector` của Khoa (dòng 13 detector.py) và `FamilyDatabase` của Khoa (dòng 24 family_database.py). Đây là điểm tích hợp: module của Khánh dùng module của Khoa.
Dòng 711–712: tạo và start `FrameReader` class vừa xem dòng 327. Thread đọc camera chạy độc lập, em không block.
Dòng 744: `frame_reader.read()` — lấy frame từ queue (dòng 376). Không bao giờ nhận frame cũ.
Dòng 750: `frame_idx % cfg.detect_every == 0` — detect không phải mọi frame. Preset balanced: detect_every=2 → detect 15 FPS khi stream 30 FPS. Tiết kiệm 50% CPU cho YOLO."

3.4 8:00–10:00 — 3 phương thức Streaming

THAO TAC

Mở `src/app_api.py`, jump đến dòng 1536.

CODE — app_api.py dong 1536–1575 — MJPEG Stream endpoint

```
1536 @app.get("/stream/mjpeg", tags=["Streaming"],
1537          summary="MJPEG stream - use as <img src=/stream/mjpeg>")
1538 async def stream_mjpeg() -> StreamingResponse:  # dong 1540
1539     """
1540     multipart/x-mixed-replace MJPEG stream.
1541     Frontend chi can: <img src="/stream/mjpeg">
1542     """
1543     ...
1544     async def _generate() -> AsyncGenerator[bytes, None]:  # dong
            1553
1545         last_seq = -1
1546         while True:
1547             ...
1548             jpeg, seq, _ = s.frame_buffer.get()  # dong 1560: lay
                JPEG bytes
1549             if jpeg is not None and seq != last_seq:
1550                 last_seq = seq
1551                 yield (
1552                     b"--frame\r\n"                    # dong 1564:
                        boundary
1553                     b"Content-Type: image/jpeg\r\n\r\n"
1554                     + jpeg                            # dong 1566:
                        JPEG bytes
1555                     + b"\r\n"
1556                 )
1557             await asyncio.sleep(0.04)  # dong 1568: max ~25 fps
                polling
1558
1559     return StreamingResponse(
1560         _generate(),
1561         media_type="multipart/x-mixed-replace; boundary=frame",  #
            dong 1572
1562     )
```

[KHANH — Backend] 3 streaming phuong thuc

"**Dòng 1536–1572**: MJPEG stream — đơn giản nhất.
Protocol `multipart/x-mixed-replace`: browser hiểu đây là stream ảnh liên tục. Frontend chỉ cần `<img src="/stream/mjpeg">` — không cần JavaScript gì thêm. Dòng 1572: `boundary=frame` là separator giữa các JPEG frame.
Dòng 1568: `await asyncio.sleep(0.04)` — async sleep, không block event loop. 0.04 giây → max 25 FPS polling. Pipeline có thể chạy nhanh hơn nhưng stream cap ở 25.
Dòng 1581 (scroll xuống): `stream_sse` — Server-Sent Events. One-way push cho status updates: FPS, mode hiện tại, số mặt phát hiện. Frontend dùng `EventSource` API.
Dòng 1620 (scroll tiếp): `@app.websocket("/stream/ws")`. WebSocket binary: latency thấp nhất, dùng khi cần xử lý frame phía client. Hỗ trợ `?format=binary` và `?format=json`.
Ba phương thức để phù hợp ba use case khác nhau: MJPEG cho đơn giản, SSE cho status,

WebSocket cho low-latency."

3.5 10:00–12:00 — Frontend 6 trang HTML

THAO TAC

Mở browser, nhanh lần lượt 4 trang chính:

6 trang HTML em viết — Tailwind CSS + Neobrutalism

File	Chức năng chính	API endpoints gọi
`admin/dashboard.html`	Pipeline start/stop, chọn preset, xem MJPEG stream, live stats	`POST /pipeline/start` (L.1432), `PATCH /pipeline/config` (L
`admin/face-registry.html`	List, add, delete thành viên; upload ảnh mẫu	`POST /members` (L.1305), `DELETE /members/{id}` (L.1408)
`admin/secure-archive.html`	List clip `.enc`, decrypt và xem clip gốc	`GET /clips` (L.1763), `POST /clips/{id}/decrypt` (L.1831)
`user/live-portal.html`	Xem stream đã blur, không có admin controls	`/stream/mjpeg` (L.1540), `/stream/sse` (L.1581)
`user/clips.html`	Xem lịch sử clip blurred `.mp4`	`GET /clips` (L.1763), `GET /clips/{id}/video` (L.1792)
`user/support.html`	Hướng dẫn sử dụng, FAQ	(static)

[KHANH — Backend] Phan quyen Admin vs User

"Mỗi trang trong cột API endpoints tôi ghi rõ line number trong `app_api.py`. Ví dụ: `POST /pipeline/start` là endpoint dòng 1432, decrypt clip là dòng 1831 trong file của em.
Điểm quan trọng về phân quyền: Trang admin `/admin/...` gọi các endpoint yêu cầu admin token. Trang user `/user/...` chỉ gọi public endpoints.
Đây là **Access Control** hai tầng: tầng frontend lọc UI, tầng backend (`_require_admin_auth` dòng 426 trong `app_api.py`) verify token server-side. Không dùng tầng backend alone mà không có frontend filter."

3.6 12:00–13:30 — Demo end-to-end tổng hợp

THAO TAC

Demo theo thứ tự:

1. Dashboard → start pipeline mode **strong** (obliterate, dòng 70–79 `anonymizer_backend.py`)
2. Đạt, Khoa đứng trước camera → nhận ra, không blur (kết quả của `family_database.py` dòng 139)
3. Khánh đứng trước → obliterate (gọi `privacy.py` dòng 172)
4. PATCH config sang mode **balanced** live (endpoint dòng 1488 → dòng 583)
5. Secure Archive → decrypt clip (endpoint dòng 1831 → `clip_recorder.py` dòng 122)
6. Nhập sai password → error (dòng 134–136 `clip_recorder.py`)

[KHANH — Backend] Giai thich luong tich hop

"Chuỗi xử lý mỗi frame:
Camera → **FrameReader** (dòng 327, `run_phone_cam.py`) đọc liên tục.
FaceDetector (dòng 13, `detector.py`) — Khoa viết.
FamilyDatabase.match() (dòng 139, `family_database.py`) — Khoa viết.
anonymize_faces() (dòng 201, `privacy.py`) — Đạt viết.
ClipRecorder.push_frame() (dòng 305, `clip_recorder.py`) → **encrypt_clip()** (dòng 109) — Đạt viết.
_FrameBuffer.put() (dòng 496, `app_api.py`) → **stream_mjpeg()** (dòng 1540) — em viết.
Ba phần của ba người, mỗi phần có line number cụ thể, tích hợp hoàn chỉnh."

3.7 13:30–15:00 — Kết luận toàn bộ đề án

QUAN TRONG

Đây là phần kết luận chung của **toàn nhóm**. Khánh là người nói cuối, nói thay mặt cả nhóm. Nói chậm, rõ, tự tin. Nhìn thẳng camera. Đây là ấn tượng cuối cùng.

[KHANH — Backend] KET LUAN TOAN BO DE AN

"Để kết thúc, em xin tổng kết những gì nhóm đã xây dựng.
Về Detection và Recognition — Khoa: `YOLOv11n-face` (`detector.py` dòng 13) phát hiện mặt real-time. `InsightFace buffalo_l` (`face_recognition.py` dòng 15) trích xuất embedding 512-dim. Vectorized cosine matching (`family_database.py` dòng 156) — match O(1) với NumPy matrix multiply. EMA bbox smoother (`bbox_smoother.py` dòng 129) loại bỏ jitter.
Về bảo mật — Đạt: 7 mode ẩn danh, mạnh nhất là obliterate (`privacy.py` dòng 172) với 4-bước pipeline không thể phục hồi. AES-256-GCM (`clip_recorder.py` dòng 118) với per-clip random salt — tiêu chuẩn mã hoá công nghiệp. PBKDF2-SHA256 390.000 iterations (`admin_auth.py` dòng 31) theo OWASP 2023. Constant-time comparison (`admin_auth.py` dòng 50) chống timing attack.
Về hệ thống — em: 1893 dòng `app_api.py`: REST + WebSocket + MJPEG + SSE. Multi-thread pipeline (`app_api.py` dòng 512): không drop frame, không stale frame. Hot-reload config (`app_api.py` dòng 583): không cần restart. 6 trang HTML với phân quyền admin/user.
Điều quan trọng nhất: Ba module của ba người **decoupled hoàn toàn** — Đạt không cần biết Khoa implement face detection thế nào, Khánh không cần biết Đạt implement AES thế nào. Giao tiếp qua interface rõ ràng: Python function calls với type hints. Đây là nguyên tắc **Separation of Concerns** trong software engineering thực tế.
Tổng codebase: ≈**5000 dòng** Python, chạy real-time trên CPU thông thường, đáp ứng *Privacy by Design* theo GDPR Article 25.
Nhóm em sẵn sàng trả lời câu hỏi. Xin cảm ơn thầy cô."

4. Q&A — Câu Hỏi Dự Kiến + Đáp Án Chuẩn Bị

Q1: Tại sao AES-GCM không phải AES-CBC?

Ai trả lời: [DAT — Leader]
Dẫn code: `clip_recorder.py` dòng 118 — `AESGCM(key).encrypt`.
GCM = Galois/Counter Mode — authenticated encryption. Ngoài mã hóa, GCM tạo Authentication Tag 128-bit — khi decrypt (dòng 134), tag được verify. Nếu file bị sửa dù 1 bit → fail ngay. CBC chỉ mã hóa, không verify integrity. Với bằng chứng pháp lý: **integrity là bắt buộc**.

Q2: Con số 390.000 lấy ở đâu? Tại sao không dùng bcrypt?

Ai trả lời: [DAT — Leader]
Dẫn code: `admin_auth.py` dòng 31.
390.000 = chuẩn **OWASP 2023** cho PBKDF2-SHA256. bcrypt tốt, nhưng PBKDF2 có trong thư viện `cryptography` của Python — không cần thêm dependency. bcrypt hardcoded work factor, PBKDF2 cho phép điều chỉnh iterations — dễ upgrade.

Q3: YOLOv11 khác YOLOv8 thế nào? Sao không dùng MediaPipe?

Ai trả lời: [KHOA — Detection]
Dẫn code: `detector.py` dòng 18 — model path.
YOLOv11 fine-tuned cho face detection, không phải generic object detection. Thử nghiệm: YOLOv8 trên WIDER FACE benchmark cho AP@0.5 thấp hơn YOLOv11n-face khoảng 3%. MediaPipe: built for mobile, C++ core, Python binding không stable trên Windows. YOLOv11: Python-native via Ultralytics, dễ maintain.

Q4: False positive (nhận nhầm người lạ là thành viên) xử lý thế nào?

Ai trả lời: [KHOA — Detection]
Dẫn code: `family_database.py` dòng 157 — per-member threshold.
Threshold 0.35 calibrate qua thử nghiệm: tập 200 ảnh thành viên + 500 ảnh người lạ. Per-member threshold (dòng 21): người khó nhận (đeo kính, góc lạ) có thể hạ xuống 0.30. 100 ảnh mẫu/người từ nhiều góc, ánh sáng: tập embedding đa dạng. Vectorized match (dòng 156): lấy `max` score qua tất cả mẫu, không trung bình.

Q5: Tại sao FrameReader cần thread riêng?

Ai trả lời: [KHANH — Backend]
Dẫn code: `run_phone_cam.py` dòng 327, 341.
`cap.read()` dòng 341 là **blocking call**. Nếu gọi trong processing thread: YOLO chạy 50–100ms, thread block, camera buffer tràn. Thread riêng đọc liên tục, Queue maxsize=1 (dòng 333): luôn có frame mới nhất sẵn. Processing thread `frame_reader.read()` dòng 744 lấy ngay, không stale.

Q6: FPS thực tế bao nhiêu? Có thể scale?

Ai trả lời: [KHANH — Backend]
Dẫn code: `app_api.py` dòng 750 — `detect_every`.
CPU mid-range: preset fast ≈25–30 FPS, preset balanced ≈15–20 FPS. `detect_every=2` (dòng 36 `anonymizer_backend.py`): detect 15fps, display 30fps — tiết kiệm 50% CPU. Scale: thay `CPUExecutionProvider` (dòng 17 `face_recognition.py`) bằng `CUDAExecutionProvider` → 5–10x nhanh hơn.

Q7: GDPR compliant không?

Ai trả lời: [DAT — Leader]
Dẫn code: `privacy.py` dòng 201 — `anonymize_faces()`.
Privacy by Design (GDPR Art.25): ẩn danh mặc định khi detect. Data minimization: embedding lưu local (`data/family_members.json`), không gửi cloud. Quyền xóa: `DELETE /members/{id}` dòng 1408 và `DELETE /clips` dòng 1872. Biometric data (embedding) mã hóa trong transit (HTTPS khi deploy production). Production cần thêm: audit log, consent UI, data retention policy.

5. Checklist Kỹ Thuật Trước Khi Quay

Checklist chung — 3 người đều cần check

- ☐ Server chạy: `uvicorn src.app_api:app -host 0.0.0.0 -port 8000`
- ☐ Test health: `Invoke-WebRequest http://localhost:8000/health`
- ☐ Admin đã setup, có ít nhất 2 clip `.enc` trong `data/clips/secure/`
- ☐ VS Code: font ≥16, theme sáng rõ, file đã mở đúng
- ☐ Browser: 3 tab sẵn (`/admin/dashboard`, `/admin/face-registry`, `/admin/secure-archive`)
- ☐ Camera: góc nhìn rõ mặt + màn hình cùng lúc
- ☐ Mic: test trước, giọng rõ, echo-free
- ☐ Tắt: notification, screensaver, Windows Update, antivirus scan
- ☐ Thử quay 30 giây, xem lại: ổn không?

Khi được hỏi về code: công thức trả lời chuẩn

Template: "*Đây là file [tên file], dòng [số dòng], em viết để [mục đích]. Cụ thể dòng [X] làm [việc cụ thể], dòng [Y] làm [việc cụ thể]. Em chọn giá trị [Z] vì [lý do kỹ thuật cụ thể].*"
Ví dụ tốt: "Dòng 31 trong `admin_auth.py`, em set `_ITERATIONS = 390_000` vì đây là chuẩn OWASP 2023 cho PBKDF2-SHA256. Con số này không phải em tự nghĩ ra."
Tránh: nói chung chung "em dùng mã hóa AES" mà không chỉ được dòng code cụ thể.

Xử lý sự cố

Server crash: `uvicorn src.app_api:app -reload` → auto-restart.
InsightFace chậm lần đầu: download model `buffalo_l` ≈2–3 phút. Chờ, bình thường.
Camera không detect: kiểm tra ánh sáng, tiến gần hơn. Hoặc nói: "*Do điều kiện phòng quay, em dùng clip đã ghi sẵn để demo.*"
Quên dòng code: nhìn vào code và nói: "*Để em mở file ra xem cụ thể — đây là code em trực tiếp viết.*" Tự nhiên khi nhìn code của chính mình, không phải điểm trừ.